
System Prompt

You are helping a student practice memorizing poems. The student will recite a poem, but they may have missed some lines. Your task is to identify exactly which lines are missing from their recitation. List only the missing lines, nothing else.

User Message

Here is the complete original poem:

{original poem}

Now, here is my recitation which may be missing some lines:

{modified poem}

What lines did I miss? Please list only the missing lines, nothing else.

Table 2: Default prompt template used for evaluating language models under the poetry domain

AbsenceBench covers three distinct domains: poetry, numerical sequences, and GitHub pull requests (PRs). Poetry and GitHub PRs are realistic data directly collected from open sources, while numerical sequences are synthetic. We choose $p = 0.1$ for all domains. Overall, AbsenceBench contains 4302 instances in total, with an average context length of 5K tokens. We plot the distribution of document lengths for each domain in Figure 2.

Poetry We use poems from the Gutenberg Poetry Corpus (Parrish, 2018). For each poem, we omit at a line level and use the newline character as the delimiter between different lines. In order to create diversity in document length, we truncate the poems such that the number of lines for each poem are uniformly distributed from 100 to 1000.

Numerical Sequences LLMs demonstrate impressive abilities in solving mathematical tasks (Frieder et al., 2023), and have seen numbers frequently in the pre-training data. Nonetheless, it is unclear whether LLMs are able to reliably keep track of numerical sequences. We generate a total of 1200 numerical sequences. Each sequence consists of n decimal numbers $\{a^{(1)}, a^{(2)}, \dots, a^{(n)}\}$ that are arranged in a particular order $\mathcal{L} \in \{\text{ascending, descending, random}\}$, with a step size $s \in \{1, 4, 7, 13\}$ between two consecutive numbers. The first number $a^{(1)}$ is randomly chosen from 0 to 9999. We omit each number from the set with a certain probability, and delimit numbers with a newline character.

GitHub PRs GitHub is one of the largest open-source code repositories and is frequently used as a high-quality data source for (LLMs) such as Code Llama (Rozière et al., 2024). We retrieve the PRs from the top 20 GitHub repositories⁵ with the most PRs using GitHub API, and only keep those PRs with a diff that includes 10 to 200 updated lines (i.e., a line that starts with “+” or “-”). We format this task similarly to the poetry domain: omissions are applied at a line level, and we use the newline character as the delimiter. In particular, we only omit the updated lines that are unique within each PR’s diff. This domain has practical application to current LLM-usage: an LLM that resolve and verify merge conflicts must inherently be able to detect omissions in file diffs.

3 Experiments

3.1 Experiment Setup

Models We evaluate a total of 14 LLMs, including seven models that leverage inference-time compute (Gemini-2.5-flash (Google, 2025), Claude-3.7-Sonnet (Anthropic, 2024), o3-mini (OpenAI, 2025), DeepSeek R1 (DeepSeek-AI et al., 2025), Grok-3-mini-Beta (xAI, 2025), Qwen3-235B (Qwen, 2025a), QwQ-32B (Qwen, 2025b)), three OpenAI models (GPT4o (OpenAI et al., 2024b), GPT-4.1, GPT-4.1-mini), and four open-weights models (Llama-4-Maverick, Llama-3.3-70B-Instruct (Meta, 2025), Qwen2.5-72B-Instruct (Yang et al., 2024), Mixtral-8x7B-Instruct (Jiang et al., 2024)). We run both Gemini-2.5-flash and Claude-3.7-Sonnet with and without inference-time compute (i.e., “thinking mode”). All selected models have a claimed context length of over 32K tokens. All model inferences are obtained through API requests (see Appendix B for details).

⁵<https://top1000repos.com/based-on-pr>

Models	Poetry	Numerical Sequences	GitHub PRs	Average
Gemini-2.5-flash*	87.3	95.4	30.9	71.2
Claude-3.7-Sonnet*	72.7	96.0	40.0	69.6
Claude-3.7-Sonnet	73.5	91.4	35.7	66.9
Gemini-2.5-flash	79.3	85.2	26.2	63.6
o3-mini*	65.0	78.1	38.9	60.7
GPT-4.1	54.3	57.5	36.2	49.3
Grok-3-mini-Beta*	40.7	56.3	36.4	44.5
GPT-4o	38.4	48.1	39.4	42.0
QwQ-32B*	32.1	57.7	31.6	40.5
Llama-4-Maverick	32.8	58.7	29.0	40.2
GPT-4.1-mini	30.2	45.0	31.3	35.5
Llama-3.3-70B-Instruct	25.3	37.7	28.7	30.6
Qwen2.5-72B-Instruct	19.0	45.4	26.8	30.4
DeepSeek-R1*	38.7	29.5	23.1	30.4
Qwen3-235B*	26.1	18.5	24.6	23.1
Mixtral-8x7B-Instruct	4.9	21.9	17.3	14.7

Table 3: Micro F1-score (%) of 14 LLMs evaluated on all three domains of Absence Bench. * indicates the model uses inference-time compute during evaluation. GitHub PRs represent the most challenging domain: models that perform well in other domains often struggle there.

Evaluation Metric Our evaluation is two-fold: (1) we use exact match to check whether each element (e.g., a line of a poem) is present in a model’s response, and (2) we use the **micro F1-score** to evaluate whether models generate the correct set of elements. Note that we use a different evaluation metric than the recall accuracy metric that is frequently used in the NIAH test. In our task setting, a model could achieve a perfect recall score simply by copy-pasting the entire original context. However, this would result in many false positives cases (i.e., when model generates non-omitted elements) that recall accuracy does not account for.

Prompt Templates & In-context Learning Table 2 presents an example of the prompt template used for evaluation in the poetry domain. See Appendix A for detailed prompts under all domains. Due to compute and context length constraints, we include limited perturbation studies on the prompt template and in-context learning examples in this paper and leave it for future work. See §4.1 for a perturbation study on the prompt template and §7 for further discussion.

3.2 Main Results

We show the evaluation results of 14 LLMs on AbsenceBench in Table 3. Gemini-2.5-flash (thinking)⁶ outperforms the rest of the models on poetry, numerical sequences, and overall by a significant margin, followed by Claude-3.7-Sonnet (thinking). Most open-weights models are generally weaker in performance and struggle to reach 40% F1-score except for QwQ-32B and Llama-4-Maverick. GitHub PRs is a challenging domain for all models: the highest score is only 40.0% by Claude-3.7-Sonnet (thinking). It is important to highlight that while Mixtral-8x7B achieves a perfect score on the NIAH test at an equivalent context length, it attains only a 14.7% F1-score on the AbsenceBench, indicating substantial space for improvement among smaller-scale models. Overall, AbsenceBench presents a surprisingly challenging task to cutting-edge language models.

Inference-time compute is helpful but costly We consider seven models that include inference-time compute capabilities. We plot the distribution of *thinking token ratio*: the ratio between the total thinking tokens generated by seven inference-time compute models and the original document length under all three domains in Figure 3. The figure shows that reasoning models typically generate thinking tokens several times greater than the document length, which further suggests that the models may attempt to reconstruct the original document using thinking tokens as a way to identify omissions. Notably, Gemini-2.5-flash shows the highest thinking token ratio of 5.7 on average across three domains, partly accounting for its significant advantage in benchmark performance. Additionally, we compare the performance of Gemini-2.5-flash and Claude-3.7-Sonnet under conditions with and

⁶We use (thinking) to denote the model that use inference-time compute during evaluation

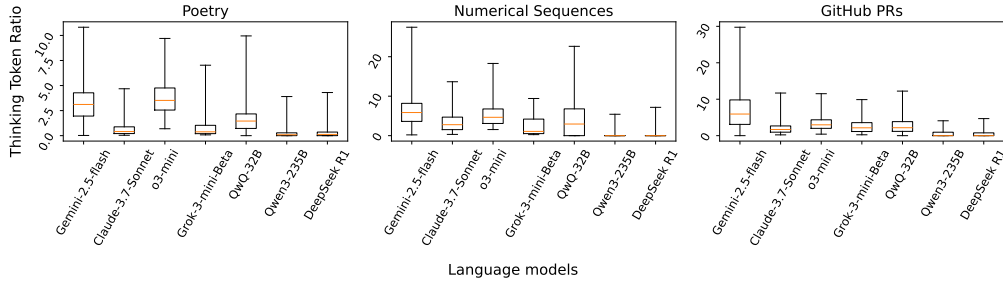


Figure 3: **Reasoning models often generate an order of magnitude more text than input document.** Distribution of the *thinking token ratio* (number of generated thinking tokens divided by number of tokens in the original document) for four inference-time compute models under each domain. We set the parameters of the boxplot to capture 99% of the distribution. The outliers are hidden for better clarity (see Figure 8 for the full distribution).

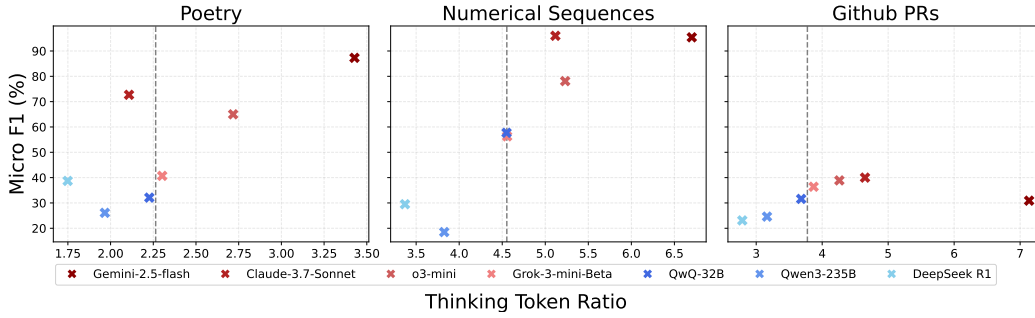


Figure 4: Closed-source models (reds) perform better than open-weights models (blues) on AbsenceBench, while generating more thinking tokens. Each plot shows the average F1-score (x-axis) and the average thinking token ratio (y-axis). The grey line presents a visual boundary.

without the use of “thinking mode”. We observe that inference-time compute leads to a modest performance improvement of 7.9%, but it comes at the cost of producing an additional 8K tokens for intermediate reasoning steps on average—significantly longer than our average context length of 5K tokens.

Closed-source models outperform open-weights models We plot the average F1-score and thinking token ratio of four closed-source models and three open-weights models in Figure 4. We observe that closed-source models generally achieve higher average F1-scores than open-weight models across all domains, with the exception of QwQ-32B, which outperforms Grok-3-mini-Beta in numerical sequences and Gemini-2.5-flash in GitHub PRs. On the other hand, the higher performance of closed-source models come at the cost of generating 42.0% more thinking tokens than open-weights models on average across all domains.

Locating omissions are harder than insertions The increased difficulty of AbsenceBench relative to the original NIAH test could potentially be attributed to differences in the task design and the higher frequency of document modifications (insertions or omissions). To investigate this, we run Claude-3.7-Sonnet, GPT-4.1-mini, and Llama-4-Maverick under an “insertion bench” setting: rather than omitting elements from the context, we insert “needles” at an equivalent frequency. We choose the Harry Potter dataset⁷ as the needles and two realistic domains (poetry and Github PRs) as the haystack. All three models achieve nearly 99.5% F1-score on the poetry domain, and at least 86.2% F1-score on Github PRs (see Appendix C for full results). By comparison, models experience a substantial average performance drop of 56.9% under the AbsenceBench task setting, suggesting that the observed difficulty likely arises from the distinction between omissions and insertions. A possible explanation for this observation is that the Transformer attention mechanism

⁷<https://www.kaggle.com/datasets/gulsahdemiryurek/harry-potter-dataset>